

A Study of Documentation in Agile Software Projects

Stefan Voigt

Otto von Guericke University/ Fraunhofer IFF Magdeburg
39106 Magdeburg, Germany
+49 391 4090 713
stefan.voigt@ovgu.de/
@iff.fraunhofer.de

Jörg von Garrel

SRH FernHochschule Riedlingen
88499 Riedlingen, Germany
+49 152 22628050
joerg.vongarrel@hs-riedlingen.de

Julia Müller, Dominic Wirth

Fraunhofer IFF
39106 Magdeburg, Germany
+49 391 4090 - 0
julia.mueller@iff.fraunhofer.de,
dominic.wirth@iff.fraunhofer.de

ABSTRACT

Although agile methods have become established in software engineering, documentation in projects is rare. Employing a theoretical model of information and documentation, our paper analyzes documentation practices in agile software projects in their entirety. Our analysis uses method triangulation: partly-structured interviews, observation and online survey. We demonstrate the correlation between satisfaction with information searches and the amount of documentation that exists for most types of information as an example. Also digital searches demand nearly twice as much time as documentation. In the conclusion, we provide recommendations on the use of support methods or tools to shape agile documentation.

CCS Concepts

• Software and its engineering, agile software development • Software and its engineering, documentation

Keywords

Information behavior; agile documentation

1. INTRODUCTION

Agile software development methods play an important role in research and practice [1]. “Traditionalists” and “agilists” have debated the correct interpretation of values and principles of the Agile Manifesto [6]. Very often the second value “Working software over comprehensive documentation” is misunderstood [4]. Some agilists even view documentation as a waste of time since it does not contribute to the final product [14]. Agile methods and documentation are not mutually exclusive, though [7, 15]. Even agile development requires a certain amount of documentation [7, 16] and even agile developers consider documentation to be an important issue [17]. This makes it essential to obtain more detailed information from past projects on what developers need and how their needs can be met. A complete picture of the state of documentation and software developers’ concrete needs will help us determine how to optimize documentation. A theoretical model of information and documentation behavior serves as the basis for an empirical analysis that delivers insights into the state-of-the-art and needs in information seeking and documentation. Limitations of length restrict

our examination to a portion of a larger analysis.

This paper is structured as follows. A review of the literature on agile software development and documentation in agile environments is given in section 2. Section 3 presents a theoretical model of the relations between information needs and seeking on the one hand, and information use and documentation, on the other. Section 4 describes the methodological approach used in our research. The results of the empirical analysis are presented in section 5 and discussed in section 6. Conclusions about the development of methods and tools that support documentation are drawn from the analysis and presented in section 7.

2. REVIEW OF LITERATURE ON AGILE DOCUMENTATION¹

The software industry has played a key role in the development and dissemination of agile methods. Demand for agile research was great fifteen years ago. The most recent overview of the state-of-the-art is provided by Dingsør et al. [4]. Their study reveals that research on agile methodologies is quite developed. Knowledge management is one of the most frequently employed theoretical approaches to agile research. This is interesting because the accumulation of knowledge in documentation is one important aspect of knowledge management. Research tends to focus more on knowledge generation in agile teams, though. The authors also address the frequently encountered misunderstanding that agile methods do not require any documentation. [4]

Literature on agile documentation exists as well but it only touches on specific aspects of documentation and does not provide complete solutions (e.g. [7, 9, 12, 15, 21, 22]). Empirical research on agile methods in general and documentation in agile projects in particular is relevant to this paper. Although Dybå and Dingsør demonstrated that a good empirical basis for agile software development now exists [5], very few empirical studies on agile documentation have actually been published:

Hoda et al. present the documentation strategies employed in agile projects in [8]. A survey of fifty-eight developers from twenty-three companies identified five documentation strategies, including electronic backups of printed artifacts and documentation of change requests or business terms used by clients.

Stettina et al. [18] and Stettina and Kroon [19] examined groups of students to analyze documentation routines in agile teams and project delivery. The teams analyzed organized their work themselves and usually put the worst developer in charge of documentation. In the surveyed sample, documentation was perceived to be burdensome but knowledge transfer was considered

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ESEM '16, September 08 - 09, 2016, Ciudad Real, Spain

Copyright is held by the owner/author(s). Publication rights licensed to ACM.

ACM 978-1-4503-4427-2/16/09...\$15.00

DOI: <http://dx.doi.org/10.1145/2961111.2962616>

¹ We used the search terms “agile” or “agility” and “documentation” or “document” to identify relevant literature in Google Scholar, ACM Digital Library, IEEE Explore, Springerlink.

to be important. [18] Second study also revealed that documentation is neglected very often. Since they fail to see any added value, development teams do not like to write documentation. The authors identified three parameters of documentation: scope of project, time pressure, commitment to documentation. [19]

In-depth insight into agile software engineers' needs for information that can be covered by documentation is still lacking.

3. A MODEL OF INFORMATION AND DOCUMENTATION BEHAVIOR

Choo developed a general model of information behavior based on a variety of research studies of established information behavior [2]. One main advantage of Choo's model is its very general level of abstraction from concrete settings, thus remaining simple, comprehensible and consistent. Choo's model builds upon other established studies: Taylor's information use environments (IUE) [2], Dervin's sense-making model [20] and Kuhlthau's information search process [11]. His model provides a good foundation for use in concrete settings such as documentation in agile software projects for our analysis.

Choo describes three stages: information need, information seeking and information use (see Figure 1). These three stages are "[...] embedded in an information processing environment composed of the individual's internal cognitive structures and emotional dispositions, and a larger information use environment determined by the conditions of the work or social setting in which the information use takes place." [2] Choo's general model can be summarized as follows: Need for information triggers information seeking that ideally results in use of the information found. The sequence of the three stages is dynamic (a recursive sequence being possible) and is influenced by the individual's cognitive needs (Dervin [20]) and affective reactions (Kuhlthau [11]). These cognitive and affective influences are dictated by information use environments (Taylor [2]). Due to limitation of length, our paper focuses on the sequence of the three stages of information needs, information seeking and information use/documentation (see Figure 1) and does not focus on IUE, cognitive needs or affective responses.

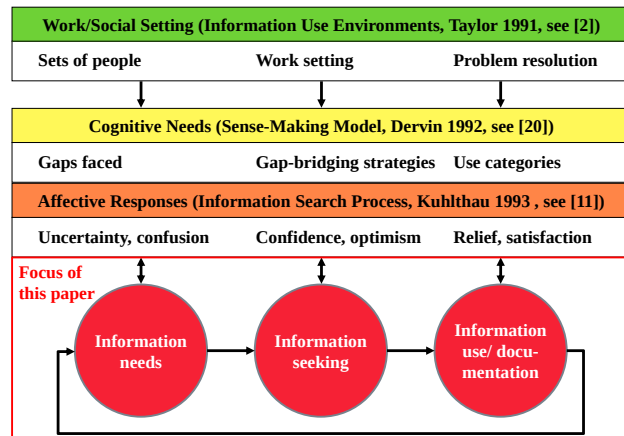


Figure 1. Information and documentation behavior model adopted from Choo's model [2]

Simply put, documentation is one possible use of information for this application and is thus covered by Choo's model. The recursive adaptability of the stages of need, seeking and use are particularly suited for documentation: Documentation gives rise to certain needs for information, which have to be satisfied by seeking in existing information systems.

4. METHODOLOGICAL APPROACH

Our paper is an outgrowth of the sprintDoc research project (www.sprintdoc.de) and is intended to develop a set of tools and methods for agile documentation. We surveyed the current state and concrete demands in the analysis phase. In addition to Otto von Guericke University, a software development company working on the documentation tool prototype and two pilot user companies that will test the developments, are involved in this project. Both pilot user companies were analyzed here.

4.1 Methods used and research questions

We applied methodological triangulation to obtain different perspectives on the subject of study with the following methods:

- nine partially structured, guided interviews with software developers from the pilot user companies, which included a brief questionnaire to self-assess the time requirements for different tasks,
- participatory observation with job analysis at three enterprises, each with two developers in parallel, for a working week,
- an online survey, with 104 fully answered responses from German, mostly agile software development SMEs to verify the results of our pilot user companies analysis².

The following questions (Q) and hypotheses (H) guided the analysis:

Q1: What percentage of time is spent on information seeking and documentation in agile software projects? Since documentation is often negated in agile practice [14], the amount of time spent on documentation can be assumed to be relatively low and might be lower than the amount of time spent on active information seeking. We focus on digital information seeking here because our documentation tool will support information seeking, too³. This leads us to *H1: More time is spent on digital information seeking than documentation.*

Q2: How does documentation work in agile projects? The concrete content of documentation and its relation to the need for information are primarily of interest. *H2.1: Primarily requirements and bugs (e.g. tasks/issues) and source code are documented in agile software projects.* Since agile approaches focus on the continuous delivery of working software, source code is often the only artifact documented [15]. Moreover, requirements are usually recorded in user stories [8]. *H2.2: Agile companies rely on direct communication instead of documentation.* This hypothesis is based on the Agile Manifesto and on empirical studies e.g. [19]. *H2.3: Existing forms of documentation (except source code documentation) do not cover the need for information.* If we assume that there is only little documentation (see H2.1), then we can infer that this documentation does not suitably cover the overall need for information.

Q3: What concrete information do developers need from past projects? Since documentation is only expedient when it can cover (future) need for information, it is important to identify this need. *H3 Information on other content (e.g. architecture*

² The online-survey was distributed via email to German SME developing software. Only fully answered questionnaires were used for analysis, this led to a responding rate of 1.2%.

³ We are aware that conversation may be a type of information search, too. Since it is difficult to determine from observation whether a conversation is a response to an information need or merely small talk, we treated every conversation identically.

information, lessons learned) than requirements, bugs and source code (see H.2.1) is needed and sought more often. Since software projects differ depending on their application domain (context), we assume that the contextual needs vary widely.

4.2 Description of pilot user companies

We describe the two pilot user SMEs in order to render the IUE more understandable. SME1 works in the fields of industrial communication and distributed automation. One third of its thirty-five employees work on the development and testing of software for process and factory automation, one of its main business segments. Custom applications are usually interface specifications compliant with different industrial standards. SME1 relies on a combination of typical iterative software development and modern agile approaches common in German software companies [10].

SME2 is a provider of IT services for Internet and cloud applications and electronic commerce. The company sells standard web-based products. Customization is usually done agilely (scrum) but sometimes in a combination of conventional collaboration with clients and internal agile implementation.

Our analysis in both SMEs entailed asking managers and software developers to use a grid to assess the past two year's projects (n=29). Most projects were organized using a combination of traditional and agile approaches (58%) but only 21% of them were purely agile (something common in Germany [10]).

Although our prototype development company did not participate in the interviews, it did take part in our observational study to help provide a broader base of data. This company employs nearly twenty software developers who mainly produce Internet and smartphone applications for their customers. Most projects are managed agilely using scrum.

4.3 Online survey sample

In addition to observing pilot users, we also surveyed German software companies online (n=99)⁴. Over half of the respondents (57%) rank software engineering as their company's main business activity. The majority of the respondent companies develop custom software applications (81%) or proprietary software (65%). Two thirds of the companies are SMEs (fewer than 250 employees). Approximately two thirds of the respondents are directly involved in software projects. Roughly half of respondents have up to fifteen employees working on software development. Average team size is between five and six members. Agile approaches play an important role in software projects in the sample, 84% of the respondents ranking their importance as "high" and "very high". The approach used most frequently is scrum (70%), followed by internal, individual approaches (38%). Roughly half of the respondents have more than five years of experience with agile software development.

5. RESULTS OF EMPIRICAL ANALYSIS

Our work is based on Choo's model analyzing the three stages of information need, information seeking and information use/documentation in agile software projects.

5.1 Information need

Respondents to the online survey were asked which of the types

of information⁵ listed are important to them. Over one third indicated that "project overview with goal" (44%), "specific characteristics/lessons learned" (37%), "knowledge about the application domain" (37%) and "overall picture as architecture/design model" (35%) are very important information. The combined mentions ranked as "important" and "very important" deliver the following descending ranking: "identified bugs" (82%), "project overview with goal" (80%) and "solutions and guidance" (79%). Only about two thirds consider "test information" (66%) or "concrete solution fragments (source code)" (63%) to be (very) important information (see Figure 2).

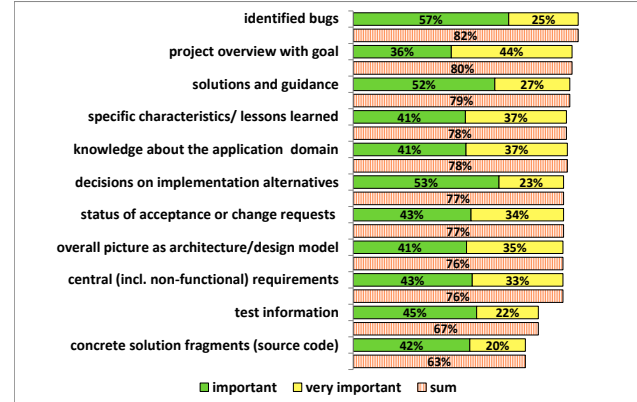


Figure 2. Importance of different information⁶

5.2 Information seeking

In the interviews, we asked how much work time is spent on information seeking and documentation, the question on time spent on information seeking and later documentation being a stand-alone question in the interviews (see A in Table 1). At the end of the interviews, every developer was given a brief questionnaire in which all tasks were listed and the work time consumed evaluated (see B in Table 1). Interestingly, the mean of the answers was three times higher in the interviews (A) than in the brief questionnaire (B).

Table 1. Comparison of assessments of time spent seeking information⁷

Interview part	Estimations of Interviewees in %											⊙
A) stand-alone	10	10	6	50	25	20	50	20	2,5	/	/	22
B) questionnaire	5	5	5	25	5	5	1	8	2	5	15	7

The online survey also asked how much time was spent seeking information. The mean was 24%. As in the interviews, the question in the online questionnaire was a stand-alone question and responses were very similar to the assessments in the interviews.

These individual assessments were objectivized using an activity analysis app for tablets. We observed two developers in each of the three companies (our two pilot user companies and the development partner) for a period of one work week. Once we

⁴ 104 fully completed surveys: We excluded five because they ranked the importance of agile approaches as "very low" (or "do not know").

⁵ The types of information are derived from the literature on documentation in agile software projects (e.g. documents suggested or information type focused on in the studies).

⁶ Scale: unimportant, less important, important, very important

⁷ Interviewees were not required to answer every question. The two developers from SME3 were not interviewed.

eliminated the categories “personal/break” and “other” from our data, a total time of 197:38:23 hours and 3,499 tasks remained.

Remarkably, nearly a quarter of all work time is spent on “conversation” (22%) followed by “bug fixing” (18%), “testing including preparation” (14%) and “coding” (12%). All other tasks account for less than 10% of work time. “Meetings” (8%) and “information seeking” (7%) are comparable. “Organizational matters including project management” and “digital communication” make up 4%. A small percentage is spent on “documentation”, “review”, “maintenance and system support” (each 3%) and “telephone calls” (1%). Figure 3 compares the self-assessment and the objective measurement of time spent on software development tasks at the three SMEs.

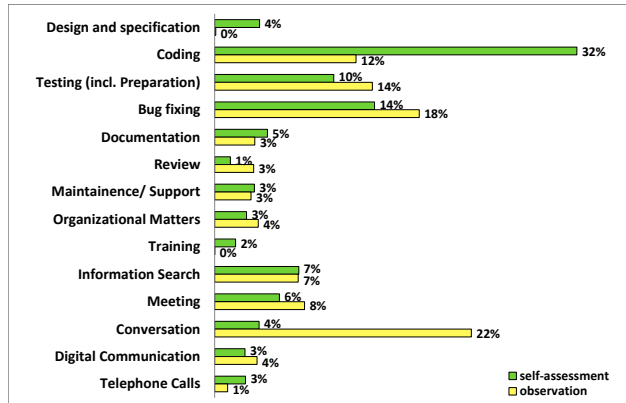


Figure 3. Comparison of average self-assessment⁸ and cumulative observation of the time spent on different tasks

5.3 Information use and documentation

5.3.1 Time spent on documentation

Developers spent 3% of their work time on documentation (see Figure 3). The mean response to this stand-alone question was 8% in the interviews (A) and 5% in the brief questionnaire (B in Table 2). The latter is comparable to the observation.

Table 2. Comparison of assessments of time spent on documentation

Interview part	Estimations of Interviewees in %											⊖
A) stand-alone	/	10	5	13	5	9	7,5	5	6	/	/	8
B) questionnaire	5	5	5	10	5	3	1	5	1	7	4	5

The mean response to the stand-alone question on documentation in the online questionnaire was 16%, thus double the mean of the self-assessment among pilot user companies and triple the observationally measured value.

5.3.2 Quantity of documentation

A combined 84% of the respondents to the online survey consider their documentation of tasks, tickets and issues to be “acceptable” and “very detailed”. This is followed by a combined 69% (“acceptable” and “very detailed”) for project meeting documentation. Software documentation generates the least documentation with a combined response of 57% for “rudimentary” and “nonexistent” (see Figure 4).

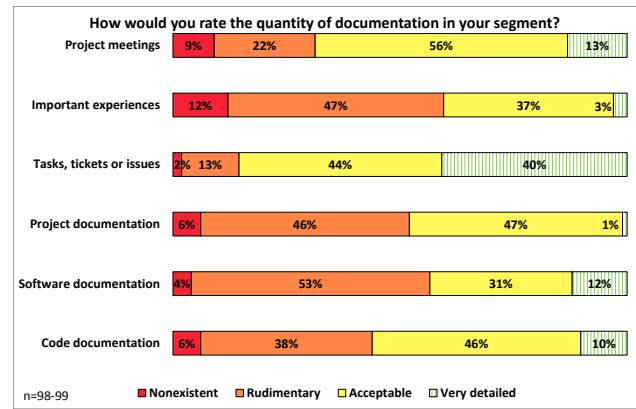


Figure 4. Quantity of documentation from the online survey

5.3.3 Correlation of information needs and documentation

As part of our online survey, we conducted a detailed analysis of the (A) importance of information needs, (B) the satisfaction with information searches and (C) the level of documentation of particular informational content (see Figure 5).

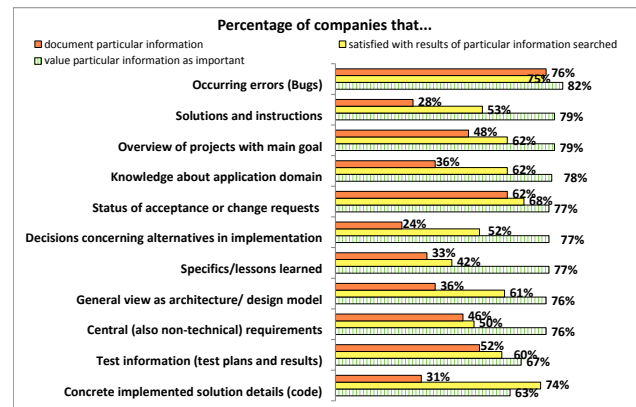


Figure 5. Importance of and satisfaction with search results and level of documentation⁹

The majority of the online survey respondents (63-82%) rank all informational content as (highly) relevant (see 5.1). Satisfaction with information searches is more diffuse. Nearly three quarters of respondents (74% and 75%) are very satisfied or rather satisfied with “details of concrete implemented solutions (code fragments)” and “occurring bugs”. Only 42% of the respondents are satisfied with the results of information searches on “specifics or lessons learned in a project”. About half of the respondents rank information on “central (as well as non-technical) requirements” and “decisions concerning alternatives in implementation” (50% and 52%) as satisfactory. Documentation is even more varied. Only a small group of respondents document comprehensive information on “decisions concerning alternatives in implementation” (24%), “solutions and instructions” (28%) and “details of concrete implemented solutions (code fragments)” (31%). The “status of acceptance or change requests

⁸ Self-assessment entailed completing a short questionnaire during the interviews (see section 5.2).

⁹ We dichotomized the four-point scales of importance (“very important”, “important”, “less important” and “unimportant”), satisfaction (“very satisfied”, “satisfied”, “less satisfied” or “unsatisfied”) and quantity of documentation (“almost complete”, “much”, “little” or “none”).

of functionality” is documented almost completely by 62% of the respondents, the highest percentage of documentation.

We also performed a correlation analysis to analyze the respondents’ awareness of the significance of documentation for satisfactory information searches. Table 3 reveals that a statistically significant correlation exists between the level of documentation and the satisfaction with particular information (except information on “solution and instructions” and “details of concrete implemented solutions (code fragments)”).

Table 3. Analysis of the correlation between the level of documentation and satisfaction with information searches

Degree of Documentation	Satisfaction with Information		
	Correlation (Pearson)	Significance (two sided)	N
Overview of projects with main goal	.226*	.034	88
Central (as well as non-technical) requirements	.365**	.000	90
General view as architecture/design model	.516**	.000	88
Specifics/lessons learned	.345**	.001	87
Decisions concerning alternatives in implementation	.300**	.004	89
Status of acceptance or change requests of functionality	.369**	.000	88
Knowledge about application domain	.498**	.000	83
Test information (test plans and results)	.415**	.000	84
Solutions and instructions	.141	.179	93
Concrete implemented solution details	.135	.217	85
Occurring errors (bugs)	.337**	.001	95
** Correlation is significant (two sided) at a level of 0.01 or * at a level of 0.05.			

6. DISCUSSION

6.1 Hypothesis and research questions

Q1: What percentage of time is spent on information seeking and documentation in agile software projects? The resulting times vary depending on the method used. A more objective approach renders the values measured by the task analysis more realistic. 7% was measured for information seeking and 3% for documentation. Responses to the self-assessment increased as much as threefold when only one activity was inquired about (stand-alone). The self-assessed values related to other tasks were lower and matched the measured values. We believe that “documentation” and “information seeking” are both tasks with negative connotations and are thus estimated high. Developers adjust their originally high estimates to their correlations of tasks. Hypothesis H1 is confirmed. Twice as much time is spent on information seeking than on documentation. A different study concluded that 32% of work time is spent on information seeking [3]. The combined values of 7% for (digital) information seeking and 22% for conversation responding to information needs to an unmeasured degree totals 29%, which is quite close to 32%.

Q2: How does documentation work in agile projects? H2.1: Primarily requirements and bugs (as e.g. tasks/issues) and source code are documented in agile software projects. The

combined responses of “very detailed” and “acceptable” reveal that 84% of the survey respondents consider tasks to be well documented. Two thirds consider project meetings to be well documented, whereas 56% state that for source code. Less than half feel this way about software structure (43%) or important experience (40%). This confirms this hypothesis as relates to requirements. Since project meetings are documented more than source code, so the second part of H2.1 must be rejected.

H2.2: Agile companies rely on direct communication instead of documentation. Our findings support this hypothesis derived from theory. The most frequently observed task was “conversation” (22%). Another paper assigns values even higher than 30% to interaction but it is unclear whether the software projects were conventional or agile and which activities constituted “interaction” [13]. Our combined values of conversation and other types of communication deliver a very similar value (35%). Documentation only takes up 3% of work time.

H2.3: Existing forms of documentations (except source code documentation) do not cover the need for information. Between 63% and 82% of the respondents assign (dichotomized) importance to different types of information, i.e. at least two thirds of the respondents indicate that information apart from “details of concrete implemented solutions (code fragments)” is important or even very important. 24% to 76% of the respondents indicated that they have much or almost complete documentation but only three of the eleven types of information listed got more than 50% of responds for “much” or “almost complete” documentation. Consequently, there is not that much documentation for most of the types. On the other hand, the respondents indicated that they are more satisfied than unsatisfied with the results of searches for ten out of eleven types of information. At least two thirds are satisfied with the results of searches for three types of information. Thus, the type of information has to be identified exactly. It is impossible to assert that developers are generally unable to satisfy their information needs. The largest discrepancy between importance and satisfaction pertains to the following four types of information in descending order: “specifics/lessons learned”, “solutions and instructions”, “central (as well as non-technical) requirements” and “decisions concerning alternatives in implementation”. The respondents are generally aware of the significance of documentation for information seeking. This does not unconditionally mean that a high level of documentation of information generates high satisfaction with information seeking. Instead, the particular information content has to be taken into account.

Q3: What concrete information do developers need from past projects? H3: Information on other content (e.g. architecture information, lessons learned) than requirements, bugs and source code (see H.2.1) is needed and sought more often. The need for information cannot be generalized or tied to a few types of information. The dichotomized importance of types of information differs little. At least two thirds of all respondents consider all information apart from “details of concrete implemented solutions (code fragments)” to be important or even very important. The type of information most important to the respondents to our online survey is “occurring errors (bugs)”, which 82% ranked as important or very important.

6.2 Constraints

Working together with pilot user companies that got funding affected our analysis positively: The developers at pilot user companies were budgeted time to give interviews and to be observed for one week. A wider empirical basis would be desir-

able. Even though we were glad to receive more than 100 fully completed questionnaires, some 400 would have been better and enabled us to achieve a sampling error of 5% instead of around 10% (65,000 companies in the ITC service sector in Germany). While we are aware of the inherent problems of an online survey targeting an unknown population, we are convinced that our study improves our understanding of information needs and documentation behavior in agile software projects.

7. CONCLUSION

Our study demonstrated that documentation contributes to satisfaction with the results of information searches. Given the compatibility of agile approaches and lean documentation, we see a need for research on methods and tools that facilitate agile documentation. Based on our findings, we have formulated the following recommendations and requirements.

Since some information (e.g. issues) is quite well documented, documentation should concentrate on the gaps, e.g. software documentation (architecture, design model) or important experiences. Above all, it should intelligently include existing information without duplication. A documentation method ought to provide some rules specifying what, when, how and how much should be documented without torpedoing agile values.

We suggest analyzing any organization or team before deciding which information is worth documenting for future projects. The importance should harmonize with the quantity of existing documentation and satisfaction with search results.

Information seeking and documentation should be combined. On the one hand, this will mean focusing on the information needed by users (see above). On the other hand, different information pools will have to be integrated. Information from other sources should be linked and not maintained in duplicate. A best case scenario would cover all of this with one tool.

Though it may appear to be incompatible with agile approaches at first glance, the importance of documentation has to be stressed and addressed. Investing slightly more time in documentation activities could help satisfy future needs for information. Additionally, this could cut the amount of time spent learning and working, which would be duplicated otherwise.

8. ACKNOWLEDGMENT

This study was funded by the German Federal Ministry of Education and Research (BMBF) in its program “Innovative SMEs: ICT” and overseen by the DLR Project Management Agency for Information Technology [01IS15005A-D].

9. REFERENCES

- [1] Abrahamsson, P., Babar, M. A., and Kruchten, P. 2010. Agility and Architecture: Can They Coexist? *IEEE Softw.* 27, 2, 16–22.
- [2] Choo, C. W. 1998. *The knowing organization. How organizations use information to construct meaning, create knowledge, and make decisions.* Oxford Univ. Press, N.Y.
- [3] de Souza, Sergio Cozzetti B., Anquetil, N., and Oliveira, K. M. de. A study of the documentation essential to software maintenance. In *the 23rd annual international conference on Design of communication documenting & designing for pervasive information - SIGDOC '05*, 68.
- [4] Dingsøyr, T., Nerur, S., Balijepally, V., and Moe, N. B. 2012. A decade of agile methodologies. Towards explaining agile software development. *Journal of Systems and Software* 85, 6, 1213–1221.
- [5] Dybå, T. and Dingsøyr, T. 2008. Empirical studies of agile software development. A systematic review. *Information and Software Technology* 50, 9-10, 833–859.
- [6] Glass, R. L. 2001. Agile versus traditional: Make love, not war! *Cutter IT Journal* 14, 12, 12–18.
- [7] Hadar, I., Sherman, S., Hadar, E., and Harrison, J. J. 2013. Less is more: Architecture documentation for agile development. In *2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, 121–124.
- [8] Hoda, R., Noble, J., and Marshall, S. 2012. Documentation strategies on agile software development projects. *IJAESD* 1, 1, 23.
- [9] Kanwal, F., Bashir, K., and Ali, A. H. 2014. Documentation Practices for Offshore Agile Software Development. *Life Science Journal* 10, 11, 70–73.
- [10] Komus, A. (2015): Studie Status Quo Agile 2014. Ergebnisbericht. www.status-quo-agile.de, 20 June 2016.
- [11] Kuhlthau, C. C. 2004. *Seeking Meaning: A Process Approach to Library and Information Services.* Libraries Unlimited.
- [12] Myklebust, T., Stålhaneb, T., Hanssena, G. K., Wienc, T., and Haugseta, B. 2014. Scrum, documentation and the IEC 61508-3:2010 software standard. In *PSAM12 Proceedings*.
- [13] Perlow, L. A. 1999. The Time Famine: Toward a Sociology of Work Time. *Administrative Science Quarterly* 44, 1, 57.
- [14] Prause, C. R. and Durdik, Z. 2012. Architectural design and documentation: Waste in agile development? In *2012 International Conference on Software and System Process (ICSSP)*, 130–134.
- [15] Rubin, E. and Rubin, H. 2011. Supporting agile software development through active documentation. *Requirements Eng* 16, 2, 117–132.
- [16] Selic, B. 2009. Agile Documentation, Anyone? *IEEE Softw.* 26, 6, 11–12.
- [17] Stettina, C. J. and Heijstek, W. 2011. Necessary and neglected? an empirical study of internal documentation in agile software development teams. In *SIGDOC '11 Proceedings of the 29th ACM international conference on Design of communication*, 159–166.
- [18] Stettina, C. J., Heijstek, W., and Faegri, T. E. 2012. Documentation Work in Agile Teams: The Role of Documentation Formalism in Achieving a Sustainable Practice. In *2012 Agile Conference*, 31–40.
- [19] Stettina, C. J. and Kroon, E. 2013. Is there an Agile Handover? An Empirical Study of Documentation and Project Handover Practices Across Agile Software Teams. In *19th International ICE-IEEE ITMC Conference on Engineering, Technology and Innovation (ICE)*.
- [20] Tidline, T. J. 2006. Dervin's Sense-Making. In *Theories of information behavior*, K. E. Fisher, S. Erdelez and McKechnie, L. E. F., Eds. ASIST monograph series. Information Today, Medford, NJ, 113–117.
- [21] Tripathi, V. and Goyal, A. K. 2014. A Document Driven Approach for Agile Software Development. *International Journal of Advanced Research in Computer Science and Software Engineering (IJARCSSE)* 4, 4, 1085–1090.
- [22] Uikey, N., Suman, U., and Ramani, A. K. 2011. A Document Approach in Agile Software Development. *International Journal of Software Engineering (IJSE)*, 2, 13–22.